

String Matching Algorithms Based upon the Uniqueness Property

Chia-Wei Lu and R. C. T. Lee

Department of Computer Science and Information Engineering

National Chi Nan University, Nantou, Taiwan, ROC

cwlu@alg.csie.ncnu.edu.tw, rctlee@ncnu.edu.tw

Abstract

In this paper, we consider the exact string matching problem which is to determine all of the locations of a pattern string appearing in a text string. We first point out a uniqueness property of a given pattern. We then propose three some algorithms to solve the exact string matching problem based upon the uniqueness property. Experimental results showed that two of our algorithms are faster than the KMP and Boyer-Moore algorithms. For a text of 250000 characters with a vocabulary of four characters a pattern with 8000 characters, one of our algorithms took 1.412ms to solve the exact matching problem. For the same data, another algorithm took only 1.06ms.

Introduction

In this paper, we are concerned with the exact string matching problem in which we are given a pattern $P = p_1p_2\dots p_m$ and a text $T = t_1t_2\dots t_n$, $n \geq m$, and we are asked to find all occurrences of P in T . Much research has been done in the area [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 and 15].

In almost all algorithms, we open a window in T with size m and try to see whether P matches with the substring in this window. A straight forward approach is to compare the substring in the window with P character by character from left to right. If a mismatch is found, move the pattern one step to the right. This operation is equivalent to opening a new window and we may repeat the above describe operation again and again. This approach is an exhaustive search approach because every substring is compared.

Nearly all exact string matching algorithms try to avoid exhaustive search. In the following, we shall introduce a rule, called the suffix to prefix rule, which allows us to avoid an exhaustive search.

Section 1. The Suffix to Prefix Rule

Let us consider Figure 1-1. Suppose we have longest suffix u of a window which is also a prefix of P , we can move P in such a way that the prefix u of P matches with the suffix u of the window.

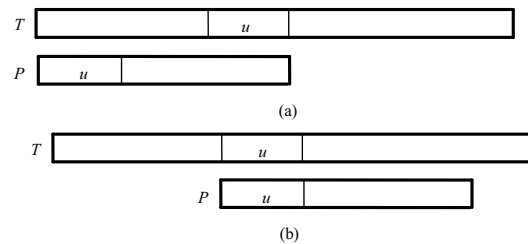


Figure 1-1

Many string matching algorithms use this suffix to prefix rule [3, 5, 6, 11, 12, 13 and 14]. This suffix to prefix rule is a general principle. It may be used together with other rules. For instance, the window doesn't have to be of size m . A smaller window may be used. Besides, as shown in Figure 1-1, it would be desirable if u is very small because a small u would enhance a long movement of P .

In the next section, we shall introduce a new concept, based upon the uniqueness property, which will make the suffix to prefix rule a very efficient rule when it is used.

Section 2. The Uniqueness Property of a String

Consider a pattern P . In P , let us find a substring which occurs in P only once. For instance, for $P = \text{CATAGTAGCCT}$, there are many such substrings. Suppose we use the substring CC and in some instance, CC in P matches CC in T exactly, as shown in Figure 2-1.

T : CTAGCGTAGACCATAG...

P : CATAGTAGCCT

Figure 2-1

We now may apply the suffix to prefix rule and move P many steps to the right, as shown in Figure 2-2.

T : CTAGCGTAGACCATAG...
 P : CATAGTAGCCT

Figure 2-2

Why is this uniqueness property significant? Let us denote the unique substring by V . Since V is unique, there is no prefix in P equal to V . As shown in Figure 2-3, if there is a suffix u in T which is equal to a prefix in P , that u must be shorter than V , which is desirable.

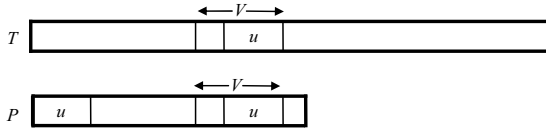


Figure 2-3

Section 3. Algorithm 1-The Longest Prefix with Unique Suffix Matching Algorithm

In this section, we shall introduce our first string matching algorithm based upon the uniqueness property. We further modified the uniqueness by noting that the substring does not have to be unique in the entire pattern P . In fact, a substring which is unique in a prefix of P suffices. Therefore, we only have to find the longest prefix which contains a unique suffix in P . For example, if $P = \text{CACTAGCCACTCTC}$, we may use TC as our selected substring. Note that TC occurs twice in P , but it is unique in the prefix CACTAGCCACTC.

As shown in Figure 3-1, if TC is used, we could move P by 11 steps to the right.

T : CTAGCGTATGCCAGTCACGATCGAGCAGGCTAC...
 P : CACTAGCCACTCTC
 P : CACTAGCCACTCTC

Figure 3-1

In the above instance, the substring G is also unique in the prefix CACTAG and in some instance, G in P matches G in T exactly, and we could move P by 6 steps to the right as shown in Figure 3-2.

T : CTAGCGTATGCCAGTCACGATCGAGCAGGCTAC...
 P : CACTAGCCACTCTC
 P : CACTAGCCACTCTC

Figure 3-2

In this instance, using the substring TC appears to be better than the substring G. But, in general, the probability of TC in P matching with TC in T exactly is $1/16$ (Suppose the size of alphabet is 4), and the probability of G in P matching with G in T exactly is $1/4$. Thus, if the substring TC in P exactly matches with TC in T once and moves P by 11 steps, the substring G in P may match G in T four times and move P by 6 steps for each time. So, we expect that the substring G would be better than the substring TC in general.

We now define a ratio to determine which substring is better.

$\sigma = (\text{Steps of moving } P) / |\Sigma|^{(\text{Size of substring})}$. The larger σ is, the better efficiency can be achieved in the searching phase.

In the preprocessing phase of this algorithm, it would determine the substring according to the ratio σ .

I. Preprocessing phase

We define a function, named *border*, and $\text{border}(V)$ would return the length of the longest suffix of V which is equal to a prefix of P . Let l be the number of steps of moving P . When it finds a prefix $P_{1,i}$ with a unique suffix V , $l = i - \text{border}(V)$.

For instance, for $P = \text{CACTAGCCACTCTC}$. Suppose we use the substring $P_{9,10} = \text{GC}$, and it is a unique suffix in the prefix $P_{1,10}$. $l = 10 - \text{border}(\text{GC}) = 10 - 1 = 9$.

When we find the longest prefix P_{1,i_1} with a unique suffix V_1 which occurs in P_{1,i_1} once, we will continue to find another longest prefix P_{1,i_2} with a unique suffix V_2 which occurs in P_{1,i_2} once, where $|V_2| > |V_1|$, and $(i_2 - \text{border}(V_2)) > |\Sigma|^{|V_2| - |V_1|} \times (i_1 - \text{border}(V_1))$.

In this algorithm, it would first try to find the longest prefix with a character as the unique suffix V and record the value l . After it finds the longest prefix with unique suffix which length is

$|V|$, it would update the value of l and try to find the longest prefix with unique suffix which length is $|V|+1$. It will stop when $|\Sigma|^{|V_2|-|V_1|} \times l > m$, and return the indexes of V in P , i_v, j_v and the variable l .

II. Searching phase

In searching phase, it would open a window $T_{i, i+m-1}$ ($1 \leq i \leq n-m+1$) with length m to match with P and move P from left to right. In every window $T_{i, i+m-1}$, we first match the substring V with $T_{i+i_v, i+j_v}$, and there are two cases shown as following:

Case 1:

If there is a mismatch when we match the substring V with $T_{i+i_v, i+j_v}$, we move P by one step. (Shown in Figure 3-3(a))

Case 2:

If V matches with $T_{i+i_v, i+j_v}$, we match the other positions of P with $T_{i, i+m-1}$, and no matter whether there is a mismatch occurring or not, we can move P by l steps. (Shown in Figure 3-3(b))

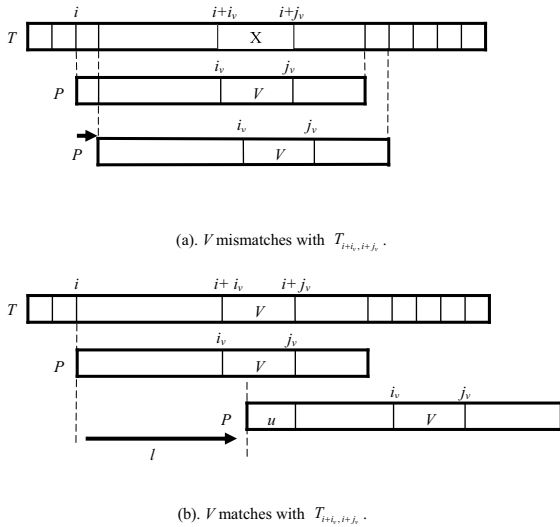


Figure 3-3. Searching phase of Algorithm 1.

III. Pseudo code

Input: P and T .

Output: The positions which P occurs in T .

```
/* Preprocessing phase */
for all  $c \in \Sigma$ 
    tmp[c] = 0;
```

```
for(i=0; i<m; i++){
    if(tmp[pattern[i]] == 0){
         $i_v = j_v = i$ ;
         $l = i$ ;
    }
    tmp[pattern[i]] = i;
}
length=1;
k = 1;
while( $|\Sigma|^k \times l < m$ ){
    for(i=m-length; i>=0; i--){
        if(  $P_{i, i+length}$  is unique in
             $P_{1, i+length}$  ){
             $i_v = i$ ;
             $j_v = i + length$ ;
             $l = i + length$ ;
            -border( $P_{i, i+length}$ );
            k=0;
            break;
        }
    }
    length++;
    k++;
}
```

/* Searching phase */

```
for(i=0; i<|T|-|P|; i++){
    Step 1: Match  $P_{i_v, j_v}$  with  $T_{i+i_v, i+j_v}$ .
    if (  $P_{i_v, j_v}$  mismatch with  $T_{i+i_v, i+j_v}$  )
        Move  $P$  one step and go to Step 1.

    Step 2: Match  $P$  with  $T_{i, i+m-1}$  (ignore
         $P_{i_v, j_v}$  )
    if (  $P$  mismatch with  $T_{i, i+m-1}$  )
        Move  $P$  by  $l$  steps and go to Step 1.

    else output  $i$ , and move  $P$  by  $l$  steps.
}
```

In the best case, the substring V is p_m . Every time in case 2 of searching phase, we can move P by m .

In the worst case, the substring V is p_1 , that is, all the characters of P are the same, and we could always move P only one step in the searching phase.

Section 4. Algorithm 2-Longest Substring with Unique Character

Matching Algorithm

Consider Figure 4-1. In the window, let x be any character. In order to have any meaningful matching of P with T , we must find the same x in P located in the left side of x in T , as shown in Figure 4-1(a). Then we move P to the right as shown in Figure 4-1(b).

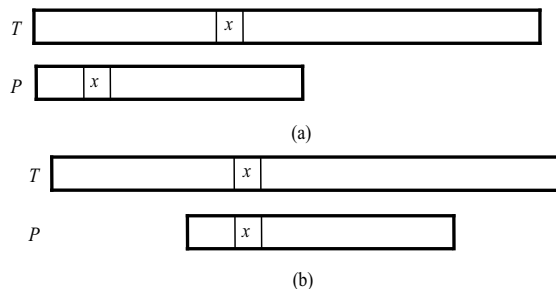


Figure 4-1

Let us now see how the uniqueness property can help. Consider the longest substring p' in P such that x in p' occurs only once. That is, $p' = p_i \dots p_j$ and p_j occurs in p' only once, as shown in Figure 4-2. Note that in the case of Figure 4-2 (b), $p_{i-1} = p_j$.

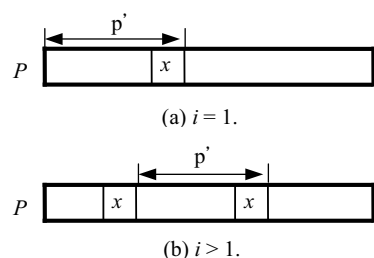


Figure 4-2

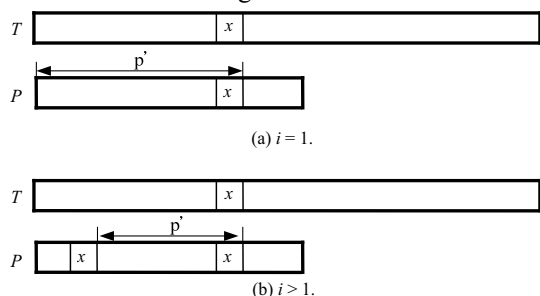


Figure 4-3

If there is matching as shown in Figure 4-3, then we can move P to the right as shown in Figure 4-4.

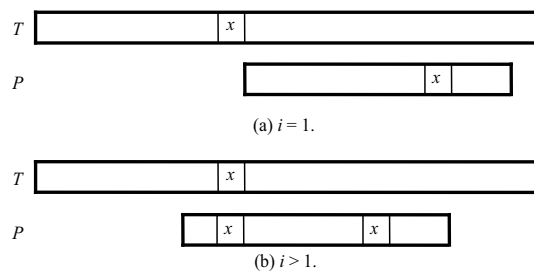


Figure 4-4

I. Preprocessing phase

Given a pattern P , we find the longest substring of P , $P_{i,j}$, where p_j is unique in $P_{i,j}$. Let l be the length of $P_{i,j}$. We return the position j and the value l for searching phase.

II. Searching phase

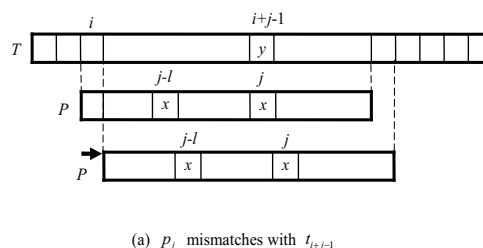
When we search the positions where P occurs in T , we open a window $T_{i,i+m-1}$ with length m to match P and move P from left to right. When we match P with $T_{i,i+m-1}$, we first match the character p_j with t_{i+j-1} , and take actions according to the two cases shown in the following:

Case 1:

If p_j mismatches with t_{i+j-1} , we move P by one step. (Shown in Figure 4-5(a))

Case 2:

If p_j matches with t_{i+j-1} , we match the other positions of P with $T_{i,i+m-1}$, and no matter whether there is a mismatch occurring or not, we can move P by l steps. (Shown in Figure 4-5(b))



(a) p_j mismatches with t_{i+j-1}

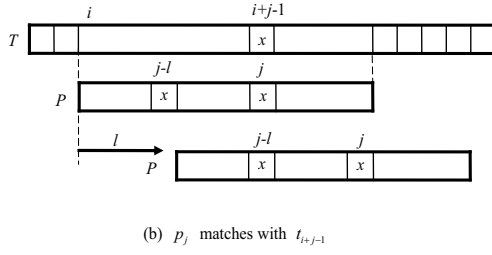


Figure4-5

III. Pseudo code

Input: P and T .

Output: The positions which P occurs in T .

/* Preprocessing phase */

```

for all  $c \in \Sigma$ 
    tmp[c] = 0;
for(i=0; i<m; i++){
    if(tmp[pattern[i]] == 0){
        l = i;
        j = i;
    }
    tmp[pattern[i]] = i;
}
for all  $c \in \Sigma$ 
    tmp[c] = 0;

for(i= m; i>0; i--){
    if(tmp[pattern[i]] - i > l){
        l = tmp[pattern[i]] - i;
        j = i;
    }
    tmp[pattern[i]] = i;
}
Go to Searching phase

```

/* Searching phase */

```

for(i=0; i<|T|-|P|; i++){
    Step 1: Match  $p_j$  with  $t_{i+j-1}$ .
    if ( $p_j$  mismatches with  $t_{i+j-1}$ )
        Move  $P$  one step and go to Step 1.

    Step 2: Match  $P$  with  $T_{i,i+m-1}$  (ignore
         $p_j$ )
    if ( $P$  mismatches with  $T_{i,i+m-1}$ )
        Move  $P$  by  $l$  steps and go to Step 1.
    else output  $i$ , and move  $P$  by  $l$  steps.
}

```

The preprocessing can be done in linear time, and it has the better efficiency in searching time

than Algorithm 1.

Section 5. Algorithm 3-The Unique Pairwise Substring Algorithm

Let us consider the case when $P = \text{GCGCACTCGTCT}$. There are many unique substrings in this pattern. For instance, CGC is a unique substring of P . But, there is another substring, namely CTC, which is quite similar to it, and we shall later explain this special uniqueness property. In the following, we first give this kind of substring precise definition.

Given a string X , a substring $x_i x_{i+1} \dots x_{j-1} x_j$ is called a unique pairwise substring if it satisfies the condition that $x_i x_{i+1} \dots x_{j-1} x_j$ occurs in the prefix $x_1 x_2 \dots x_{j-1} x_j$ of X exactly once for any $x_{i+1} \dots x_{j-1}$.

For example, if $P = \text{CACTCAGCCACTCGC}$, we may use TCG as our selected pairwise substring. Note that TCG is unique in CACTCAGCCACTCG and there is no other TxG occurring in CACTCAGCCACTCG for any x . In some instance, the characters T and G of the substring match the characters T and G of TxG in T exactly for any x , and we could move P by 12 steps as shown in Figure 5-1.

T : CAGGGAGCTAGCTAGGTAGACTACGATCGATGCCCGCAT...

P : CACTCAGCCACTCGC

P : CACTCAGCCACTCGC

Figure 5-1

I. Preprocessing phase

There are many such unique pairwise substrings, and this algorithm will select the one which is located at rightest in the pattern. Record the positions of first and last character of the selected substring as i_v and j_v . Let l be equal to i_v . Return i_v , j_v and l .

II. Searching phase

When we search the positions where P occurs in T , we open a window $T_{i,i+m-1}$ with length m to match P and move P from left to right. When we match P with $T_{i,i+m-1}$, we first match the characters p_{i_v} with t_{i+i_v-1} , p_{j_v} with t_{i+j_v-1} , and there are two cases shown in the following:

Case 1:

If p_{i_v} mismatches with t_{i+i_v-1} or p_{j_v} mismatches with t_{i+j_v-1} , we move P by one step. (Shown in Figure 5-2.(a))

Case 2:

If p_{i_v} matches with t_{i+i_v-1} and p_{j_v} matches with t_{i+j_v-1} , we match the other positions of P with $T_{i,i+m-1}$, and no matter whether there is a mismatch occurring or not, we can move P by l steps. (Shown in Figure 5-2.(b))

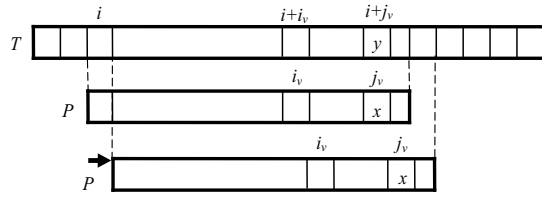
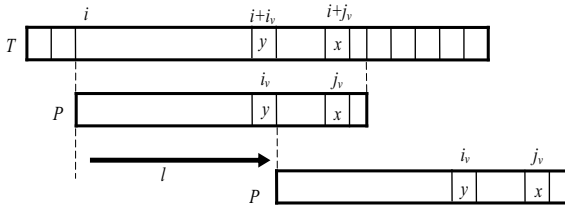
(a) p_{i_v} or p_{j_v} mismatches(b) p_{i_v} and p_{j_v} matches

Figure 5-2

III. Pseudo code

Input: P and T .

Output: The positions which P occurs in T .

```

/* Preprocessing phase */
for( $i = m-l; i > 0; i--$ ) {
    for( $j = i+1; j \leq m; j++$ ) {
        if(Substring  $p_i p_{i+1} \dots p_{j-1} p_j$  are
           unique in  $P_{1,j}$  for any
            $p_{i+1} \dots p_{j-1}$ ) {
             $l = i;$ 
             $i_v = i;$ 
             $j_v = j;$ 
            Go to Searching phase
        }
    }
}

```

```

}

```

```

/* Searching phase */

```

```

for( $i=0; i < |T| - |P|; i++$ ) {

```

```

    Step 1: Match  $p_{i_v}$  with  $t_{i+i_v-1}$  and
             $p_{j_v}$  with  $t_{i+j_v-1}$ .

```

```

    if ( $p_{i_v}$  or  $p_{j_v}$  mismatch)

```

```

        Move  $P$  one step and go to Step 1.

```

```

    Step 2: Match  $P$  with  $T_{i,i+m-1}$  (ignore
             $p_{i_v}$  and  $p_{j_v}$ )

```

```

    if ( $P$  mismatches with  $T_{i,i+m-1}$ )

```

```

        Move  $P$  by  $l$  steps and go to Step 1.

```

```

    else output  $i$ , and move  $P$  by  $l$  steps.

```

```

}

```

The preprocessing takes lots of time, but it has wonderful efficiency in searching phase.

Section 6. Experiments

We have written programs to implement our algorithms. We show the searching time for different lengths of P and T ; each result is the average of 3000 experiments with different P 's and T 's. We generate T randomly by using four characters with length n , and we select a part of T with length m as P , where $n = \{100000, 250000\}$ and $m = \{50, 100, 500, 1000, 2000, 4000, 8000\}$. The results are shown in Table 7-1 and Table 7-2.

Table 7-1. ($n = 100,000$)

$m \setminus 10^{-3} \text{ sec}$	Algorithm 1	Algorithm 2	Algorithm 3	BM	KMP	Brute-force
50	1.758	1.180	1.341	1.559	5.397	6.815
100	1.506	1.152	0.823	1.294	4.306	7.118
500	1.026	0.830	0.589	0.957	4.289	6.414
1000	1.089	0.991	0.743	1.004	4.578	6.847
2000	0.927	0.705	0.653	0.876	4.699	6.526
4000	0.970	0.714	0.602	0.788	5.300	6.422
8000	1.074	0.725	0.503	0.834	4.338	5.778

Table 7-2. ($n = 250,000$)

$m \setminus 10^{-3} \text{ sec}$	Algorithm 1	Algorithm 2	Algorithm 3	BM	KMP	Brute-force
50	4.626	3.034	3.664	3.911	11.654	16.091
100	5.000	2.846	2.189	3.499	12.217	16.471
500	2.666	2.085	1.343	2.501	12.141	15.904
1000	2.821	2.451	1.526	2.511	12.449	15.905
2000	2.172	1.661	1.397	1.920	12.020	15.995
4000	2.169	1.573	1.218	1.828	12.479	14.878
8000	1.797	1.412	1.060	1.508	10.779	14.973

Section 7. Conclusions

In this paper, we have proposed three algorithms based upon the uniqueness property of a pattern. Experimental results show that our algorithms are quite efficient. Two of our algorithms are faster than the KMP and BM algorithms. In the future, we plan to do more research on the pre-processing stage. That is, we would like to develop good algorithms to do the pre-processing as we note that the pre-processing needed in each of our algorithms is an interesting problem in algorithm design. Furthermore, we would like to do more theoretical analysis of our algorithms.

References

- [1] Apostolico, A., Giancarlo, R., 1986, The Boyer-Moore-Galil string searching strategies revisited, *SIAM Journal on Computing* 15(1):98-105.
- [2] Apostolico, A., Crochemore, M., 1991, Optimal canonization of all substrings of a string, *Information and Computation* 95(1):76-95.
- [3] Boyer, R.S., Moore, J.S., 1977, A fast string searching algorithm. *Communications of the ACM*. 20:762-772.
- [4] Colussi, L., 1991, Correctness and efficiency of the pattern matching algorithms, *Information and Computation* 95(2):225-251.
- [5] Crochemore, M., Czumaj, A., Gasieniec, L., Jarominek, S., Lecroq, T., Plandowski, W., Rytter, W., 1992, Deux méthodes pour accélérer l'algorithme de Boyer-Moore, in *Théorie des Automates et Applications, Actes des 2e Journées Franco-Belges*, D. Krob ed., Rouen, France, 1991, pp 45-63, PUR 176, Rouen, France.
- [6] Colussi, L., 1994, Fastest pattern matching in strings, *Journal of Algorithms*. 16(2):163-189.
- [7] Charras, C., Lecroq, T., Pehoushek, J.D., 1998, A very fast string matching algorithm for small alphabets and long patterns, in *Proceedings of the 9th Annual Symposium on Combinatorial Pattern Matching*, M. Farach-Colton ed., Piscataway, New Jersey, Lecture Notes in Computer Science 1448, pp 55-64, Springer-Verlag, Berlin.
- [8] Galil, Z., Seiferas, J., 1983, Time-space optimal string matching, *Journal of*

- Computer and System Science
26(3):280-294.
- [9] Galil, Z., Giancarlo, R., 1992, On the exact complexity of string matching: upper bounds, SIAM Journal on Computing, 21(3):407-437.
 - [10] Horspool, R.N., 1980, Practical fast searching in strings, Software - Practice & Experience, 10(6):501-506.
 - [11] Knuth, D.E., Morris (Jr), J.H., Pratt, V.R., 1977, Fast pattern matching in strings, SIAM Journal on Computing 6(1):323-350.
 - [12] Lecroq, T., 1992, A variation on the Boyer-Moore algorithm, Theoretical Computer Science 92(1):119-144.
 - [13] Morris (Jr), J.H., Pratt, V.R., 1970, A linear pattern-matching algorithm, Technical Report 40, University of California, Berkeley.
 - [14] Sunday, D.M., 1990, A very fast substring search algorithm, Communications of the ACM . 33(8):132-142.
 - [15] Simon, I., 1993, String matching algorithms and automata, in in Proceedings of 1st American Workshop on String Processing, R.A. Baeza-Yates and N. Ziviani ed., pp 151-157, Universidade Federal de Minas Gerais, Brazil.